Faculdade de Tecnologia Professor Francisco de Moura – FATEC Jacareí

ABP - Aprendizagem Baseada em Projetos

2026-2

Parceiro:	UniLaunch
Contato:	Otávio Abreu dos Santos Silva contato.stackvize@gmail.com
Período/Curso:	3º DSM
Focal point:	Prof. Rodrigo Silva Peres
Kick off:	29/09/2026 às 19h

Tema do Semestre

Plataforma para Estimativa do Impacto Ambiental de Aplicações de Software

Desafio (problema)

O crescimento da computação em nuvem e da adoção de arquiteturas baseadas em microsserviços trouxe novos desafios para organizações de todos os portes. Atualmente, uma mesma empresa pode operar dezenas ou até centenas de aplicações distribuídas em diferentes regiões do mundo, cada uma consumindo recursos computacionais de maneira distinta.

Embora métricas como utilização de CPU, memória, armazenamento e rede sejam amplamente utilizadas para monitorar desempenho e disponibilidade, elas raramente são utilizadas para compreender o impacto ambiental dessas aplicações. Como consequência, gestores possuem pouca visibilidade sobre o consumo energético de seus sistemas e sobre as emissões de carbono associadas à sua operação.

Nesse contexto, surge a necessidade de uma plataforma capaz de transformar métricas de infraestrutura em indicadores ambientais, permitindo estimar o consumo energético das aplicações e sua emissão aproximada de dióxido de carbono equivalente (CO_2e). Essas informações auxiliam equipes técnicas e gestores na identificação de oportunidades de otimização, contribuindo para decisões mais sustentáveis e alinhadas às práticas de Green Computing e ESG.

O desafio consiste no desenvolvimento do GreenER, uma plataforma web capaz de monitorar continuamente aplicações de software, estimar seu consumo energético e calcular sua emissão aproximada de CO_2 equivalente (CO_2e). Para isso, serão disponibilizados dois serviços auxiliares: um Agregador de Métricas, responsável por simular um ambiente corporativo e fornecer informações sobre aplicações em execução, e um Serviço de Intensidade de Carbono, responsável por disponibilizar fatores de emissão de carbono para diferentes regiões geográficas.

Ao longo da execução do projeto, o ambiente monitorado será dinâmico. Novos serviços poderão surgir, aplicações poderão tornar-se indisponíveis ou deixar de exportar métricas, e os valores coletados variarão continuamente. Espera-se que a solução desenvolvida seja capaz de adaptar-se

automaticamente a essas mudanças, mantendo os indicadores atualizados sem necessidade de intervenção manual.

Ao final do semestre, a aplicação deverá ser capaz de apresentar informações como consumo energético estimado, emissão de CO_2 equivalente por serviço, indicadores consolidados do ambiente monitorado e comparações entre aplicações, oferecendo uma visão clara do impacto ambiental da infraestrutura analisada.

Requisitos

Requisitos Funcionais:

RF01: Descoberta de serviços

O sistema deve consultar o Agregador de Métricas para identificar os serviços disponíveis.

RF02: Monitoramento dinâmico

O sistema deve reconhecer a inclusão, remoção, indisponibilidade e retorno de serviços durante sua execução, atualizando as informações apresentadas ao usuário.

RF03: Coleta de métricas por serviço

O sistema deve consultar periodicamente o endpoint `/metrics/{id_servico}` para cada serviço disponível.

RF04: Tratamento da variação das métricas

O sistema deve considerar que os valores retornados pela API podem mudar a cada requisição.

RF05: Detecção de indisponibilidade

O sistema deve identificar e sinalizar quando um serviço monitorado não responder à consulta ou estiver indisponível.

RF06: Detecção de ausência de métricas

O sistema deve identificar quando um serviço estiver ativo, mas deixar de exportar métricas. Ou seja, está listado em `/services` mas não está enviando métricas.

RF07: Cálculo individual

O sistema deve calcular consumo energético e emissão de CO_2e para cada serviço monitorado.

RF08: Indicadores agregados

O sistema deve calcular indicadores consolidados, como consumo total, emissão total, serviços ativos e serviços indisponíveis.

RF09: Dashboard operacional

O sistema deve apresentar, para cada serviço, seu estado de monitoramento, localização, métricas disponíveis, consumo energético estimado e emissão estimada de CO_2e.

RF10: Histórico de coletas

O sistema deve armazenar o histórico das coletas realizadas para permitir análise temporal.

RF11: Atualização contínua

O dashboard deve atualizar periodicamente suas informações, sem exigir que o usuário recarregue a página.

RF12: Localização dos serviços

O sistema deve exibir país, região e, quando disponível, cidade onde o serviço está hospedado.

RF13: Visualização geográfica

O sistema poderá apresentar em um mapa a posição aproximada dos serviços para os quais a API fornecer latitude e longitude. A ausência dessas coordenadas não deve impedir a visualização das demais informações do serviço.

RF14: Ranking de impacto

O sistema deve permitir ordenar os serviços por consumo energético estimado ou por emissão estimada de CO_2e, indicando o período considerado na comparação.

RF15: Comparação entre serviços

O sistema deve permitir comparar dois ou mais serviços por métricas e indicadores ambientais referentes ao mesmo período.

RF16: Autenticação e controle de acesso

O sistema deve autenticar o usuário responsável pela configuração do monitoramento por meio de JWT. A consulta ao dashboard pode ser pública, mas a alteração dos parâmetros de monitoramento deve exigir autenticação validada no backend.

Requisitos Não Funcionais:

RNF01: Usabilidade

A interface deve ser simples, clara e responsiva, adequada ao uso em navegadores e dispositivos móveis.

RNF02: Acessibilidade da informação

Estados como “ativo”, “indisponível” e “sem métricas”, bem como mensagens de erro e alertas, devem ser identificáveis por texto ou ícones, sem depender exclusivamente de cores.

RNF03: Atualização periódica

A coleta e a atualização da interface devem ocorrer em intervalos definidos pela aplicação. O projeto deve informar esses intervalos e a data e hora da última atualização exibida ao usuário.

RNF04: Desempenho

O tempo de resposta da interface deve ser adequado para acompanhamento contínuo do ambiente monitorado.

RNF05: Tolerância a falhas

A indisponibilidade de um serviço monitorado ou de uma das APIs auxiliares não deve interromper o funcionamento da aplicação, devendo ser apresentada ao usuário de forma adequada.

RNF06: Segurança

Senhas não devem ser armazenadas em texto puro; segredos e credenciais não devem ser incluídos no repositório; e as rotas de alteração de configurações devem validar o JWT e os dados recebidos.

RNF07: Manutenibilidade e testes

As regras de cálculo e de classificação dos estados dos serviços devem estar separadas dos controllers e possuir testes unitários que cubram resultados esperados e situações de ausência ou falha de dados.

RNF08: Documentação técnica

O projeto deve conter instruções de execução, descrição da arquitetura, modelo de dados, documentação dos endpoints desenvolvidos pela equipe e orientações para configurar o acesso às APIs auxiliares.

Restrições de Projeto:

RP01: Frontend

Obrigatoriamente React com TypeScript.

RP02: Backend

Obrigatoriamente NestJS com TypeScript, com endpoints HTTP para o frontend. As funcionalidades devem ser organizadas em módulos, controllers e services, com uso de DTOs para os dados recebidos nas rotas pertinentes.

RP03: Banco de Dados

Obrigatoriamente PostgreSQL, com persistência de dados estruturada por meio de um framework ORM (*Object-Relational Mapping*) integrado ao ecossistema NestJS. O histórico de coletas e os dados necessários às consultas temporais devem ser persistidos no banco.

RP04: Organização do código

O backend deve empregar injeção de dependências e separar as responsabilidades de acesso a dados, integração com APIs externas e regras de negócio. Os cálculos de energia e CO_2e não devem ser implementados diretamente nos controllers.

RP05: Execução containerizada

A aplicação completa, incluindo frontend, backend e PostgreSQL, deve ser iniciada exclusivamente por meio de containers Docker. As instruções de execução devem constar na documentação do projeto.

RP06: MVP

O escopo deve ser compatível com o tempo do semestre, priorizando um MVP funcional com:

- Navegação completa;
- Respostas estruturadas;
- Exibição de evidências documentais.

RP07: Autenticação

A autenticação da área de configuração deve ser implementada no backend utilizando JWT. Não será aceito controle de acesso realizado apenas no frontend.

RP08: Gestão ágil do projeto

A equipe deve manter um backlog priorizado, com histórias ou itens de trabalho e respectivos critérios de aceitação. Em cada *sprint review*, deve demonstrar as funcionalidades concluídas, registrar o retorno recebido e atualizar o planejamento das entregas seguintes.

RP09: Evidências de Interação Humano Computador

A equipe deve apresentar o protótipo das principais telas, o planejamento e os resultados da avaliação de usabilidade e as alterações feitas na interface a partir dessa avaliação.
Cronograma O cliente ainda não confirmou as datas e horários
Sprint review 1 – 20/10/2026 às 19h30 Sprint review 2 – 10/11/2026 às 19h30 Sprint review 3 – 23/11/2026 às 19h30 Apresentação final – a definir, mas será no final do semestre.

Para auxiliar no desenvolvimento, o cliente forneceu os seguintes endpoints:

Greener Metrics Aggregator API - https://metrics.unilaunch.org/docs

Greener Carbon Intensity API - https://carbon.unilaunch.org/docs